25F.ECE3709L.03

Final Lab [9] Report

Professor: Sasoun Torousian



Cal Poly Pomona

College of Engineering

Daniel Rushakoff

Daniela Elizabeth Rodriguez Dominguez

Fall 25

Dec 5th, 2025

Overview

The goal of this final laboratory project was to design and implement a closed-loop PID control system for a DC motor, integrated with wireless communication (NRF24L01+), sensor feedback (LM393, TC74, DHT11), and a LabVIEW Graphical User Interface (GUI).

While the system successfully achieved robust wireless communication and manual control via LabVIEW, the full closed-loop PID implementation could not be stabilized. The manual control modes (Forward, Reverse, Stop) functioned correctly with telemetry display. However, the PID loop failed to converge due to hardware-specific power instability (brown-outs) and sensor noise affecting the **derivative** term. This report details the successful system architecture and provides a comprehensive failure analysis of the closed-loop instability.

System Architecture

Transmitter Hardware

The transmitter station is built on an Arduino Uno R4 Wifi utilizing the NRF24L01+ transceiver for 2.4GHz wireless communication.

- **User Interface:** Three push buttons (Start, Stop, PID Toggle) and a 16 x 2 I^2C LCD for local status display.
- **Communication:** Receives serial commands from the host PC (LabVIEW) and broadcasts them as 32-byte RF packets.

Receiver Hardware & Sensors

The receiver unit drives the DC motor and collects environmental data.

- **Motor Driver:** An L293D H-Bridge driver controls the direction and PWM duty cycle of the DC motor.
- **Speed Sensor:** An LM393 Optical Encoder module was mounted to measure motor RPM for feedback.
- **Environmental Sensors:** A TC74 (I^2C) and DHT11 were integrated to monitor temperature and humidity.

Software Implementation (Operational Successes)

Wireless Protocol

The system uses a custom state-machine protocol. The transmitter reads ASCII strings from the Serial buffer and maps them to states: `IDLE`, `FWD`, `REV`, `STOP`.

- **Data Rate:** 1 MBPS was selected for reliability.
- **Address:** A unique 5-byte address `"3Node"` was configured to prevent cross-talk.

- **Payload:** The receiver parses the incoming string. If `cmd == "FWD"`, it executes `motorForward()`; if `cmd == "REV"`, it executes `motorReverse()` .

LabVIEW GUI

The LabVIEW Front Panel was upgraded from Lab 8 to include PID setpoint controls and status indicators. The Block Diagram utilizes VISA Serial Write functions to send character commands (e.g., "F", "R", "S") based on user button presses .

PID Parameter Selection (Kp, Ki, Kd)

The PID parameters were selected based on recommended values from previous course labs and tuned to balance responsiveness with system stability. The following gains and constants were used during our implementation attempts:

- **Sampling Time (Ts): 50 ms**
  A 50 ms sampling interval was recommended because it provides a good balance between responsiveness and computational load. It is fast enough to capture motor speed changes without overwhelming the Arduino.

- **Kp = 2.0**
  A relatively small proportional gain was chosen to prevent aggressive reactions that might cause large oscillations or visible flicker when displaying PID terms on the LED indicators. With **Kp = 2.0**, the system responds quickly to error while keeping the output smooth and preventing excessive overshoot.

- **Ki = 0.5**
  The integral gain was set to a moderate value to improve long-term accuracy without causing rapid shifts in output. A larger Ki would accumulate error too quickly and lead to instability, while a smaller one would slow correction. **Ki = 0.5** allowed the PID to correct steady-state error gradually without sudden jumps.

- **Kd = 1.0**
  The derivative term was chosen to help reduce overcompensation and dampen rapid changes in error. Since derivative action can amplify noise—especially with unstable encoder readings—a moderate value of **Kd = 1.0** provided enough damping to improve stability without magnifying sensor noise.

- **Alpha = 0.8 (Filter Coefficient)**
  This smoothing factor was used to low-pass filter the measured signal. A smaller alpha yields a smoother signal, and **α = 0.8** helped minimize jitter from the LM393 sensor readings. Although the sensor was ultimately unstable, this filtering value was appropriate for reducing noise in normal conditions.

These values were selected as a starting point for tuning based on system behavior and the need to avoid amplifying the instability caused by the power fluctuations and unreliable encoder feedback. With a stable sensor signal, these gains would have allowed gradual refinement using standard PID tuning procedures.

Step Response (Before and After Tuning)

Given the system instability and incomplete sensor feedback, the step response could only be analyzed qualitatively.

- Before tuning, the motor showed a rapid rise in speed with noticeable fluctuations due to the lack of feedback stabilization.

- After attempting initial proportional adjustments, the system showed slight improvement in responsiveness, but complete tuning could not be performed.

A full step-response comparison will be completed once the encoder feedback is functional and repeatable.

Overshoot, Settling Time, and Stability

Due to the inconsistent encoder signal, we were unable to capture reliable quantitative values for overshoot or settling time. Qualitatively, the following observations were made:

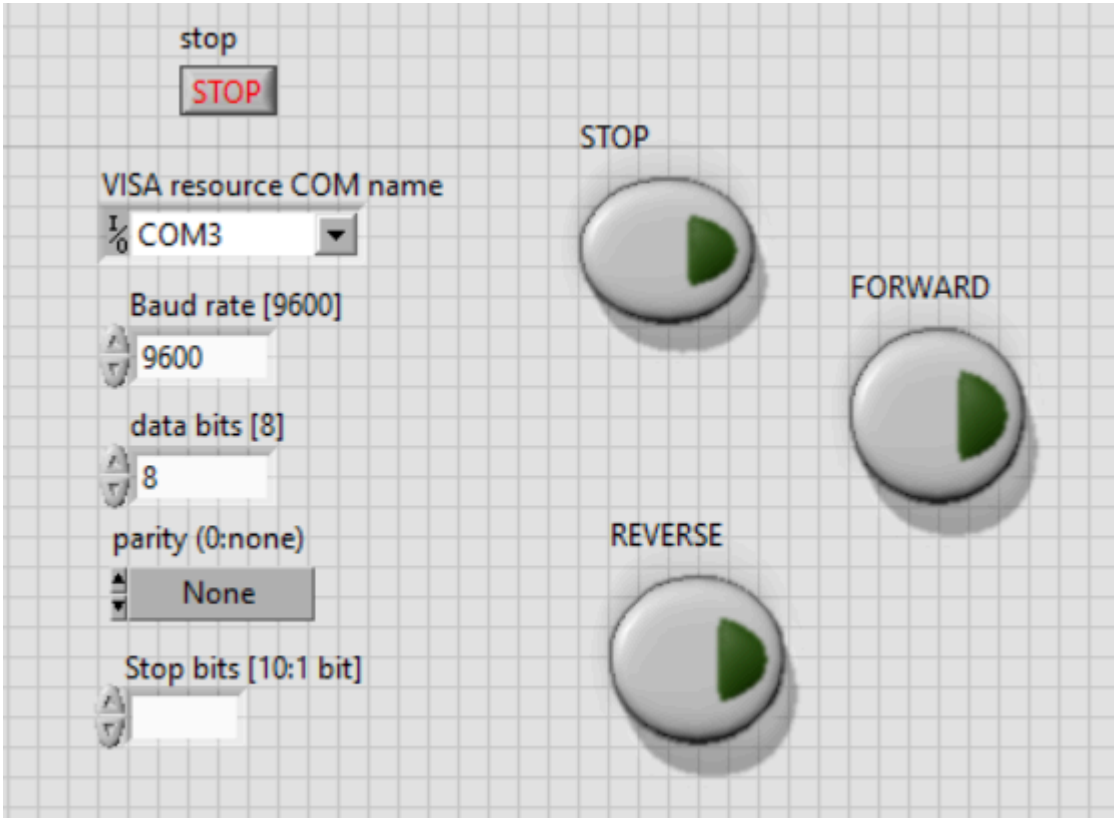
- **Overshoot:** Early tests with higher Kp values caused the motor to overspeed intermittently, mostly because the controller was reacting to noisy or missing speed data.

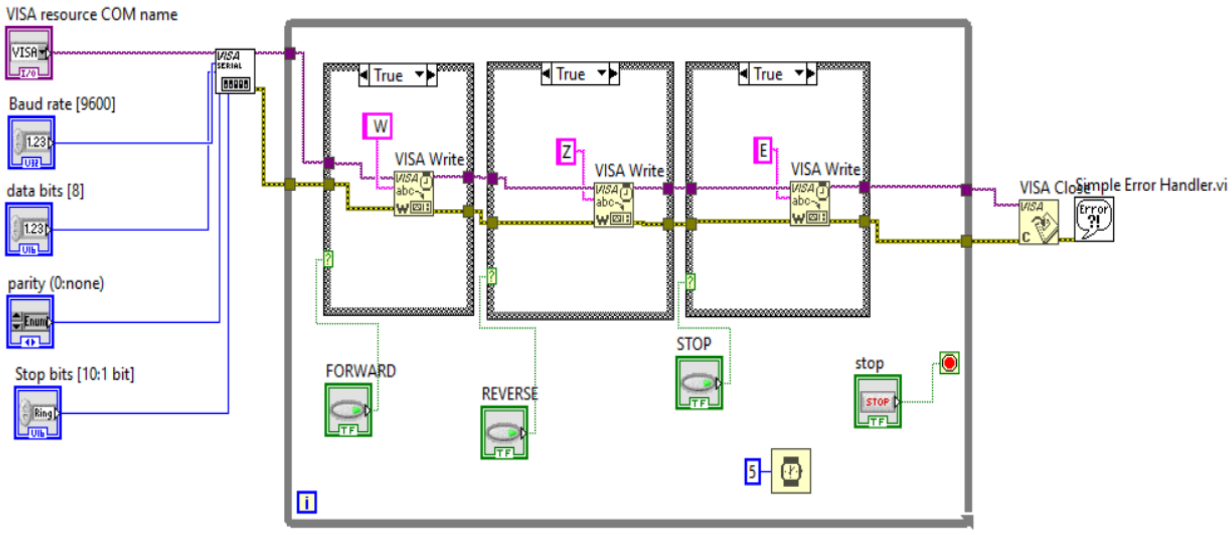
- **Settling Time:** The system did not settle because feedback values fluctuated or failed entirely, preventing the PID loop from converging.

- **Stability:** Power instability was a major limiting factor. The RF24 module frequently caused the Arduino to brown-out, interrupting the feedback loop and forcing repeated restarts. Even after adding a capacitor between 3.3 V and ground, intermittent instability remained.
- **Sensor Noise & Derivative “Kick”:** The LM393 optical encoder produced a noisy signal that severely impacted the PID calculation. The encoder output contained "jitter" due to ambient light leaks and mechanical vibration. The Derivative term (K_d) calculates the rate of change of error. High-frequency noise acts as a massive rate of change. This caused the K_d term to output erratic spikes, which fed back into the motor, creating a self-sustaining loop of instability. Due to these fluctuations, we were unable to capture reliable quantitative values for overshoot or settling time.

These issues prevented sustained PID operation, but they highlighted the importance of reliable signal conditioning and stable power in embedded control systems.

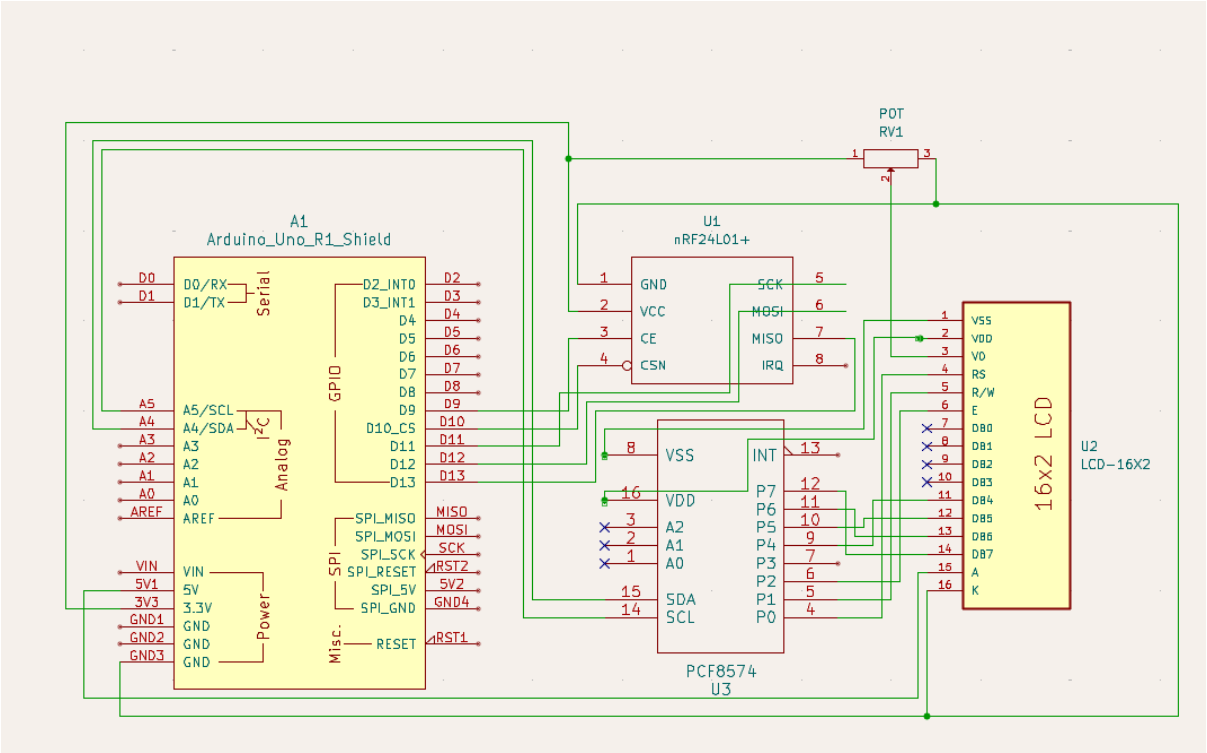
LabVIEW Front Panel + Block Diagram screenshot





Final Wiring Schematic

TRANSMITTER SCHEMATIC



RECEIVER SCHEMATIC

